

표현학습: PCA, word2vec, 임베딩의 활용

PCA · word2vec · Transformer 임베딩과 RAG

Machine Learning 1 · 14주차

한국과학영재학교 (KSA)

Chapter 14

이번 주차 개요

- **14.1 PCA** — “분산을 최대화하라”는 축을 공분산 행렬의 고유벡터로 구하고, 붓꽃 데이터를 2차원으로 압축해 본다.
- **14.2 word2vec** — bag-of-words의 한계를 넘어 skip-gram과 네거티브 샘플링으로 단어를 벡터로 만들고, “왕 - 남자 + 여자 $\approx$ 여왕”의 기하학을 들여다본다.
- **14.3 임베딩 실전** — Transformer의 첫 층도 학습된 임베딩 테이블임을 짚고, 코사인 유사도로 동작하는 RAG 검색을 손계산 $\rightarrow$ 코드로 구현한다.

이 챕터를 마치면

라그랑주 승수법으로 PCA가 고유값 문제임을 유도하고, 임베딩이 “함께 자주 나타나는 것들은 가까운 벡터로 모인다”는 한 줄 원칙을 따라 Transformer 입력층과 RAG까지 이어진다는 것을 설명할 수 있다.

14.1 PCA: 분산을 최대로 보존하는 축 찾기

정답 없이 배우다 — 비지도학습

- 지금까지의 모델은 **정답** y 가 있었다 — 집값, 스팸 여부.
- **비지도학습**은 정답이 없다. 데이터 x 자체의 **구조**를 찾는다.
- “이 고객은 몇 개 그룹으로 나뉘는가”(군집화), “1000개 특징 중 중요한 정보는 몇 차원인가”(차원축소).
- 4.3절의 **k-means**도 비지도: 라벨 없이 x 의 위치만 보고 “가까운 점끼리 묶음”.

한 줄

지도: y 를 향해 맞춰간다. 비지도: x 가 스스로 만들어내는 **패턴**을 발견한다.

왜 차원을 줄이고 싶은가 — 세 가지 이유

이유	내용
차원의 저주	특징이 100개면 kNN·k-means의 “가까움” 자체가 무의미. 중요한 축 2 3개만 남기는 것 자체가 모델을 살리는 방법.
비용·속도 시각화·해석	특징 1000개보다 중요한 성분 50개만: 학습·예측이 빠르고, 저장도 줄어든다. 4차원 이상은 사람이 못 봄. 2 3차원으로 압축해 scatter로 그리면 구조를 눈으로 판독.

세 이유는 모두 “정보를 잃되, 잃는 양을 최소화하라”로 수렴. **PCA**가 그 목표를 **분산 보존**이라는 정량적 기준으로 정리한 것.

PCA 목표와 5단계 절차

- 목표: 데이터를 저차원으로 **투영**(project)하되, **투영된 데이터의 분산이 최대인** 방향을 찾는다 = 정보를 가장 적게 잃는 방향.
- ① 평균 0이 되도록 **중심화**(centering).
- ② **공분산 행렬** $\Sigma = \frac{1}{m} X^T X$ 계산.
- ③ Σ 의 **고유벡터** · **고유값**을 구한다.
- ④ 고유값 큰 순서대로 정렬 — 각 고유벡터 = **주성분**, 대응 고유값 = 그 방향의 분산 크기.
- ⑤ 상위 d 개의 고유벡터에 투영 $\rightarrow d$ 차원 압축 표현.

투영 분산이란: $v^T \Sigma v$ 라는 한 줄

- 데이터는 이미 **중심화**되어 있다고 하자(평균 0).
- 단위벡터 v ($\|v\| = 1$) 방향으로 점 x_i 를 투영하면 투영 값은 스칼라 $p_i = v^T x_i$.
- 중심화돼 있으므로 평균이 0 $\rightarrow$ 분산은 제곱의 평균:

$$\frac{1}{m} \sum_{i=1}^m p_i^2 = \frac{1}{m} \sum_{i=1}^m (v^T x_i)(x_i^T v) = v^T \left(\frac{1}{m} \sum_{i=1}^m x_i x_i^T \right) v = v^T \Sigma v$$

$v^T x_i$ 는 스칼라라 순서를 바꿔 $v^T (x_i x_i^T) v$ 로 묶음.

결론

“방향 v 로 투영했을 때의 분산”은 $v^T \Sigma v$ 라는 **단일 스칼라**로 결정된다.

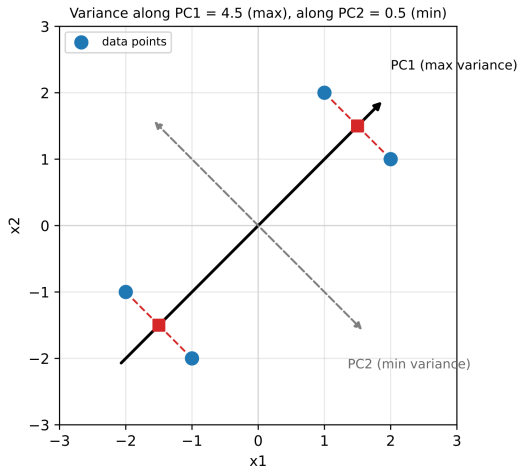
제약 문제와 투영의 그림

PCA 문제를 하나의 문장으로

$f(v) = v^T \Sigma v$ 를 최대화, 제약 $v^T v = 1$

v 를 키우면 $v^T \Sigma v$ 도 같이 커지므로, **방향만**
비교하려면 $\|v\| = 1$ 제약을 건다.

- 제약 최댓값 문제의 표준 도구: **라그랑주 승수법**.



4개 점을 분산이 최대인 PC1(45도 대각선, 분산 4.5)으로 투영. x축(분산 2.5)보다 대각선 방향이 정보를 훨씬 많이 담는다.

라그랑주 승수법 → 고유값 문제

제약 $v^T v = 1$ 을 승수 λ 로 목적함수에 붙인다:

$$\mathcal{L}(v, \lambda) = v^T \Sigma v - \lambda (v^T v - 1)$$

v 로 미분하여 0으로 놓는다 ($\frac{\partial}{\partial v}(v^T \Sigma v) = 2\Sigma v$, Σ 는 대칭이므로):

$$\frac{\partial \mathcal{L}}{\partial v} = 2\Sigma v - 2\lambda v = 0 \quad \Rightarrow \quad \Sigma v = \lambda v$$

이것이 바로 고유값 방정식

분산을 최대화하는 방향 v 는 공분산 행렬의 **고유벡터**여야 하고, 그 분산값은 대응하는 **고유값** λ 다. 최적점을 원래 목적함수에 대입하면 $v^T \Sigma v = v^T (\lambda v) = \lambda (v^T v) = \lambda$ — “그 방향의 분산 = 고유값”을 직접 확인할 수 있다.

이 유도에서 따라오는 세 가지

- **첫 번째 주성분** = 가장 큰 고유값 λ_1 의 고유벡터 v_1 . 분산이 가장 큰 방향 $\rightarrow$ 정보가 가장 많이 보존.
- **분산값 = 고유값**: i 번째 주성분으로 투영한 분산은 정확히 λ_i . “고유값 큰 순서로 정렬” = “정보량이 많은 축부터 남긴다”.
- **상호 직교성**: Σ 가 대칭 $\Rightarrow$ 서로 다른 고유값의 고유벡터는 자동으로 직교 ($v_i^T v_j = 0, i \neq j$). PC2는 “PC1과 직교하는 방향 중 분산이 큰 것” — 주성분끼리 정보 중복이 없음을 수학이 보장.

“분산을 최대화하라”는 말과 “가장 큰 고유벡터를 찾아라”는 말은 같은 문제를 다른 말로 한 것이다.

손으로 한 번: 4개 점과 공분산 행렬

의도적으로 **원점 중심**인 4개 점(중심화 단계 생략):

$$P = [(-2, -1), (-1, -2), (2, 1), (1, 2)]$$

눈으로 보면 **45도 대각선 방향으로 늘어남** — PCA의 역할은 이걸 수학으로 확인하는 것.

1단계 — 공분산 행렬 $\Sigma = \frac{1}{4} \sum_i x_i x_i^T$:

$$\Sigma = \frac{1}{4} \begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix} = \begin{pmatrix} 2.5 & 2.0 \\ 2.0 & 2.5 \end{pmatrix}$$

예: $(2, 1)$ 의 외적 — 대각에 $2^2 = 4$, 비대각에 $2 \cdot 1 = 2$. 결과 행렬은 **대칭** (공분산 행렬은 항상 대칭).

고유값과 고유벡터

2단계 — 고유값: 특성방정식 $\det(\Sigma - \lambda I) = 0$:

$$(2.5 - \lambda)^2 - 2.0^2 = 0 \Rightarrow \lambda = \frac{5 \pm 4}{2} = 4.5, 0.5$$

검산: 고유값의 합 = 대각합(trace) = $2.5 + 2.5 = 5.0 = 4.5 + 0.5 \checkmark$

3단계 — 고유벡터(주성분):

$$\lambda_1 = 4.5 : v_1 = \frac{1}{\sqrt{2}}(1, 1)^T \text{ (45도 대각선)} \quad \lambda_2 = 0.5 : v_2 = \frac{1}{\sqrt{2}}(1, -1)^T \text{ (수직)}$$

비대각 성분 $2.0 > 0$ (양의 상관) $\rightarrow$ “첫 좌표가 크면 둘째도 크다” 방향으로 기울어져 있어, 최대 분산 방향이 45도로 **기울어져 있다**.

분산을 비교하면: 왜 대각선이 이기나

4단계 — 후보 방향별 투영 분산 $v^T \Sigma v$:

방향	단위벡터 v	투영 분산
x축	$(1, 0)$	2.5
45도 대각선	$(1, 1)/\sqrt{2}$	4.5 (최대)
수직	$(1, -1)/\sqrt{2}$	0.5 (최소)
45도와 x축 사이	$(1, 0.5)/\ \cdot\ $	≈ 4.1

PCA의 본질

축을 “올바른” 방향으로 돌리면, **같은 데이터**에서 더 많은 분산(정보)을 한 축에 담을 수 있다.

$4.5 + 0.5 = 5.0$ = 원래 전체 분산 — 투영 분산의 합은 축을 아무리 돌려도 **보존**된다.

투영해 보기: 두 그룹이 완벽히 갈라진다

5단계 — $v_1 = (1, 1)/\sqrt{2}$ 에 투영: $p = v_1^T x = (x_1 + x_2)/\sqrt{2}$

$$(-2, -1), (-1, -2) \rightarrow \frac{-3}{\sqrt{2}} \approx -2.12 \quad (2, 1), (1, 2) \rightarrow \frac{+3}{\sqrt{2}} \approx +2.12$$

- 1차원 값 $\{-2.12, -2.12, +2.12, +2.12\}$ — **두 그룹이 완벽히 갈라진다.**
- 여기서 1차원 k-means를 $k = 2$ 로: 중심점 $-2.12, +2.12$, 할당 $[0, 0, 1, 1]$.
- **PCA → k-means** 파이프라인(차원 축소 → 군집화)이 작은 예시에서 제대로 작동하는 모습.

6단계 — 재구성 오차: v_1 만 남기고 재구성하면 각 점의 오차는 $|x_1 - x_2|/\sqrt{2} = 1/\sqrt{2} \approx 0.707$ (4개 점 모두 동일). 평균 제곱오차 $= 4 \times 0.707^2 / 4 = 0.5$ — **버린 주성분의 고유값 $\lambda_2 = 0.5$ 와 정확히 같음.** “상위만 남길 때 잃는 정보량 = 버린 고유값의 합”을 손으로 확인.

실습: 붓꽃 데이터를 2차원으로

iris: 꽃받침·꽃잎의 길이·너비 4개 특징, 150 샘플.

```
import numpy as np

Xc = X - X.mean(axis=0) # 중심화
cov = (Xc.T @ Xc) / len(Xc)
eigvals, eigvecs = np.linalg.eigh(cov)
order = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[order], eigvecs[:, order]

print("고유값각(축의분산):", eigvals)
print("설명 분산비율:", eigvals / eigvals.sum())

X2d = Xc @ eigvecs[:, :2] # 상위개 2 주성분에투영
```

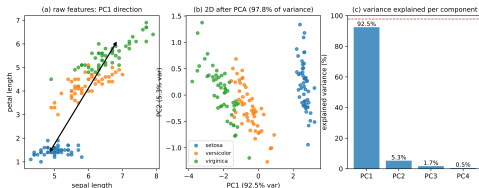
eigh: 대칭 행렬의 고유분해(단위 고유벡터 보장). 결과: 고유값 [4.200, 0.241, 0.078, 0.024], 설명 비율 [0.925, 0.053, 0.017, 0.005] — **상위 2개만으로 전체 분산의 97.8%**를 보존, 나머지 2차원을 통째로 버려도 손실 2.2%뿐.

왜 2차원으로 충분한가

- 4개 원본 특징(꽃받침 길이·너비, 꽃잎 길이·너비)은 **서로 상당히 상관되어 있다** — 예: 꽃잎이 길면 보통 넓기도 하다.
- 사실상 “독립적인 정보”는 2 3차원어치밖에 없으므로 PCA가 이 **숨겨진 중복**을 정확히 찾아 제거한다.
- sklearn 확인** (fragment: `PCA(n_components=2)`):
fit 시 자동 **중심화**, `components_`에 고유벡터,
`explained_variance_ratio_` = `[0.9246, 0.0531]` —
numpy 계산과 동일.

두 구현(numpy `eigh` vs sklearn)이 같은 결과를 내는 것 = PCA가
라이브러리 마법이 아니라 **공분산 행렬의 고유분해**라는 명확한
선형대수 연산.

가운데 패널에서 세 종류(색상)가 2차원에서도 거의 완벽히 분리 — 4차원 → 2차원으로 떨어뜨려도 “종류 구분” 구조가
거의 사라지지 않는다.



(a) 원본 공간 + PC1 방향, (b) 2차원 압축 공간
(PC1 92.5%, PC2 5.3%), (c) 설명 분산 비율 —
상위 2개가 97.8%.

실용: 몇 차원을 남길 것인가 (스크리 플롯)

- **누적** 설명 분산 비율을 그려, “더 남겨도 거의 의미 없는 작은 고유값”부터는 언제인지를 고르는 것이 표준.
- 이 누적 곡선을 **스크리 플롯**(scree plot, “벼랑”이라는 뜻 — 곡선이 급격히 떨어졌다가 평평해짐)이라 부른다.

$$\text{누적 설명 분산} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

- 가장 흔한 기준: **누적이 90%(또는 95%)를 넘는 가장 작은 k .**
- 붓꽃의 누적: [92.5%, 97.8%, 99.5%, 100%] — 1개만 남겨도 92.5%, 2개면 97.8%. “95% 기준”이면 $k = 2$.

같은 실용적 기준: 7.2절 랜덤 포레스트에서 트리 개수를 “더 늘려도 성능이 거의 안 오르는 지점”으로 정하던 것.

자주 하는 실수: 표준화 없이 PCA

- PCA가 **분산** 기준으로 축을 고르니, “수치가 큰(분산이 큰) 특징”이 자동으로 더 큰 비중을 차지.
- 특징 **단위가 서로 다르다면** 거의 항상 **버그다**.

키(cm, 평균 170, 표준편차 8)와 몸무게(kg, 평균 65, 표준편차 6) 40명:

```
raw = np.column_stack([h, wt])      # 키, 몸무게키와( 상관있음 )
cov_raw = np.cov(raw.T)              # (1) 표준화없이 PCA
w_raw, v_raw = np.linalg.eigh(cov_raw)
# RAW    PC1: [-0.851 -0.525] 차원 1 설명분산: 92.7% 키에 ( 쏠림)

scaled = (raw - raw.mean(0)) / raw.std(0)  # (2) 표준화후 PCA
cov_s = np.cov(scaled.T)
w_s, v_s = np.linalg.eigh(cov_s)
# SCALED PC1: [ 0.707  0.707] 차원 1 설명분산: 91.3% 도 (45)
```

표준화 **없이**는 PC1이 키 축에 쏠린 $\approx (-0.85, -0.53)$, **후**에는 두 축을 동등하게 대우한 $\approx (0.71, 0.71)$ 45도. **같은 데이터인데, 단위를 어떻게 줬는지에 따라 주성분의 방향이 달라진다.**

실무 규칙: StandardScaler를 먼저

- 키의 분산(≈ 64)이 몸무게(≈ 36)보다 크면, 표준화 없이 돌리면 PC1이 **거의 키 쪽에 쏠린다**
— 키와 몸무게가 실제로 얼마나 상관되는지와 **별개로**.
- 데이터가 “키가 크면 무겁다” 같은 **상관 구조**를 담고 있어도, 분산 크기의 차이가 그것을 가려버린다.
- 표준화 **후**에도 PC1이 45도(91.3%)를 유지하는 것 = “상관 구조를 정확히 찾았다”.

실무 규칙

PCA 전 **항상** StandardScaler로 각 특징을 평균 0 · 표준편차 1로 정규화.

특징들이 이미 같은 단위를 공유할 때(예: 붓꽃, 전부 cm)만 표준화 생략 가능 — **의식적으로** 생략해야지, “잊어서” 생략하는 것은 아니다.

sklearn의 PCA는 표준화를 **자동으로 해주지 않는다**(중심화만) — 스케일링은 사용자의 몫.

공분산 기준 vs 상관 기준

공분산 기준 (기본)

- 공분산 행렬 Σ 의 고유분해.
- sklearn PCA의 기본 — 중심화만 해주고, **스케일링은 안 해줌.**
- “수치가 큰 축”이 지배함.

상관 기준 (표준화 후)

- 특징을 **먼저 표준화**한 뒤 PCA.
- 고유분해의 대상이 공분산이 아니라 **상관행렬**(correlation matrix).
- 각 축이 동등한 무게를 가짐.

단위가 다른 특징이라면, 표준화 없이 돌리는 것은 버그가 아니라 버그의 원인이다. 두 기준이 언제, 얼마나 달라지는지는 앞의 키·몸무게 예가 그대로.

PCA의 한계: 세 가지 전제

PCA가 강력한 이유는 **단순함** — 그러나 그 단순함은 세 가지 전제. 전제를 벗어나면 “잘못된” 축을 반환할 수 있다.

전제	한계
직선 축만	직교한 직선(평면) 좌표계만 찾음. 원·나선·곡면처럼 비선형 으로 늘어난 데이터는 곡선을 잘라버린 직선으로 근사할 뿐. → 일반화: 커널 PCA (Ch. 5.3의 커널 트릭을 차원축소에 적용: 고차원에 올린 뒤 그 공간에서 PCA).
분산이 정보	“분산이 큰 방향 = 정보가 많은 방향”이라 정의 . 그런데 분산이 큰 방향이 라벨과 무관 일 수 있다 — 예: 이미지의 “조명 밝기”. 라벨을 몰라 구분 못 함. → 보완: 지도 차원축소 (LDA: “라벨을 구분하는 방향”을 찾음).
아웃라이어	공분산은 평균 과 2차 모멘트를 쓰므로, 멀리 떨어진 아웃라이어 하나가 공분산(따라서 고유벡터)을 그쪽으로 확 잡아당김 — 4.3절 k-means 중심점의 취약성과 같은 구조.

역사: 피어슨 → 스피어먼 → 고유얼굴

연도	인물	내용
1901	Karl Pearson	“여러 변수로 측정한 점들에 가장 잘 맞는 직선(평면)은?” — 답: 분산이 최대인 방향 . (Pearson, 1901)
1904	C. Spearman	심리측정에서 요인 분석 — 여러 능력 테스트 점수가 서로 상관된 것은, 뒤의 단일 공통 요인 (general factor, ‘g’) 덕. 사실상 요인 1개짜리 PCA 의 정서적 선구.
1991	Turk & Pentland (MIT)	얼굴을 벡터로 보고 그 공간의 주성분 방향을 고유얼굴 (eigenface)이라 부름. “눈이 가늘다/ 크다” 같은 축을 자동 추출.

새로운 얼굴을 고유얼굴에 투영한 **계수 벡터**가 곧 얼굴의 압축 표현(임베딩) — 계수 벡터 간 거리로 유사도를 잴 수 있다.

(Turk, M. & Pentland, A., 1991) — “이미지를 사람이 볼 수 있는 축으로 줄이는” 임베딩의 초기 형태.

PCA는 표현학습의 원형이다

120년의 줄기

“분산이 큰 방향을 찾아라”는 120년 전의 단순한 질문 → **얼굴 인식** → **단어 임베딩**(14.2 word2vec) → **그래프 임베딩**(Ch. 13 Node2Vec)으로 이어지는 한 줄.

- 고유얼굴: **이미지**를 벡터로 — “데이터 자체에서 구조를 찾아 저차원 벡터를 만든다”.
- word2vec: **단어**를, Node2Vec: **그래프 노드**를 벡터로 — 동일한 정신을 공유.

PCA는 표현학습(representation learning)의 가장 오래된, 가장 단순한 원형이다.

확인 문제: 1차원 데이터에 PCA

점 세 개 $x = (-2, 0, 2)$ (1차원, 이미 평균 0).

① **단계 1 — 공분산**(1차원이라 단순 분산):

$$\frac{(-2)^2 + 0^2 + 2^2}{3} = \frac{8}{3} \approx 2.67$$

② **단계 2 — 투영**: 1차원의 “방향”은 +1 또는 -1만 (± 1 이 단위벡터). +1 방향: $\{-2, 0, 2\}$,
-1 방향: $\{2, 0, -2\}$ — 부호만 바뀌니 **분산은 동일** $8/3$.

③ **단계 3 — 고유값**: 1×1 행렬 $[\sigma^2]$ 의 고유값은 그 값 자체 = $8/3$. → 단계 1의 분산과 **정확히 일치**.

요점

1차원에서는 “방향을 고르는 것”이 본질적으로 없으므로 PCA는 “데이터의 분산 자체를 돌려주는” **자명한 결과**를 낸다 — “어떤 방향으로 돌릴 것인가”는 차원 ≥ 2 에서 비로소 질문이 된다.

자주 묻는 질문 — PCA (1/2)

- **Q. 몇 개의 주성분을 남겨야 하나?** 설명 분산 비율의 **누적 합**이 90%·95% 같은 기준을 넘는 지점까지 남기는 것이 흔한 방법. (7.2절 랜덤 포레스트에서 트리 개수를 “더 늘려도 성능이 거의 안 오르는 지점”으로 정하던 것과 같은 실용 기준.)
- **Q. 정규화(L1/L2)와 비슷하지 않은가?** 목적은 비슷하나 **작동 방식이 다름** — 정규화는 학습 중 손실에 페널티를 더해 가중치를 작게 유지, PCA는 학습 **이전**에 데이터 차원을 미리 줄임(전처리). 함께 쓰는 파이프라인도 흔함: **PCA 압축** → **압축된 특징으로 정규화 모델 학습**.

자주 묻는 질문 — PCA (2/2)

- **Q. 정보는 완전히 보존되지 않는 건가?** PCA는 **손실 있는 압축**(lossy) — 상위 d 개만 남기면 나머지는 **영구히** 사라짐. 그러나 “분산 기준”으로 중요 부분부터 보존하므로, 잃는 양은 **버린 고유값의 합**으로 정확히 계산 가능 — 무작위로 잘라내는 것과 달리 “얼마나 잃는지”가 정량적.
모든 정보를 보존하려면(lossless) 전체 주성분을 모두 남겨야 한다.
- **Q. k-means 전에 항상 필요한가?** 아니다. 특히 잘 맞는 경우: (a) 특징끼리 **상관**되어 중복이 있을 때, (b) 차원이 높아 **차원의 저주**가 문제일 때, (c) **노이즈**(분산이 작은 성분) 제거가 필요할 때.
특징이 서로 무관하고 차원이 낮다면 불필요한 단계(오히려 해석성을 잃을 수 있음).

자주 묻는 질문 — PCA (3/3)

- **Q. 라벨을 모르는데, 분류 문제에도 쓸 수 있나?** PCA 자체는 라벨을 쓰지 않는다(비지도). 그러나 **전처리**로서 자주 쓰임 — “PCA 축소 → 축소된 특징으로 분류기 학습”은 표준. “라벨을 구분하는 방향”은 못 찾지만, 중복/노이즈 축을 정리하는 역할. 라벨을 아는 방법(지도 차원축소, 예: LDA)이 필요하면 다른 기법으로.
- **Q. 주성분의 방향을 “해석”할 수 있나?** 주성분은 원본 특징의 **선형 결합** — 예: “ $PC1 = 0.8 \times \text{꽃잎길이} + 0.6 \times \text{꽃잎너비} - \dots$ ”. 가중치의 **크기와 부호**(이 가중치를 **로딩**(loading)이라 부름)으로 “이 축이 어떤 원본 특징의 변동을 주로 담는지”를 판독.
“이 방향이 ‘꽃의 크기’다”처럼 의미를 부여하는 것은 **해석자의 몫** — PCA가 자동으로 “의미”를 알려주지는 않는다.

연습: 손으로 만드는 pca 함수

완성할 함수 — 중심화, 공분산, eig, 내림차순 정렬, 상위 n 개 주성분에 투영:

```
import numpy as np

def pca(X, n_components):
    Xc = X - X.mean(axis=0)
    cov = (Xc.T @ Xc) / len(Xc)
    eigvals, eigvecs = np.linalg.eigh(cov)    # 단위고유벡터보장
    order = np.argsort(eigvals)[::-1]        # 내림차순
    eigvals, eigvecs = eigvals[order], eigvecs[:, order]
    return Xc @ eigvecs[:, :n_components]

X = np.array([[ -2, -1], [-1, -2], [2, 1], [1, 2]], dtype=float)
Z = pca(X, n_components=1)
# Z = 약[ -2.12, -2.12, +2.12, +2.12] (PC1 = 도45 대각선)
# 확인: Z.var() == lambda_1 == 4.5
```

손계산 문제: 4개 점 $(-3, 0), (0, 0), (3, 0), (0, 0)$ (모두 x축 위) — (a) 공분산 행렬? (b) 고유값? (c) PC1의 방향과 투영 분산? 힌트: y축 분산은 0이므로 행렬은 $\begin{pmatrix} * & 0 \\ 0 & 0 \end{pmatrix}$ 형태 — PC1 = x축, PC2의 분산 = 0.

14.2 word2vec: 단어를 벡터로

임베딩: PCA와는 다른 방향의 압축

PCA

- “이미 **벡터인** 데이터”를 회전·압축.
- 기준: **분산 보존** (데이터 내적).

임베딩(embedding)

- 원래 벡터가 **아니었던** 것(단어, 그래프 노드)을 **학습**으로 저차원 벡터로.
- 기준: **예측 성능** (문맥 예측).

2013년 미콜로프(Tomas Mikolov) 등이 제안한 **word2vec** (Mikolov et al., 2013)은 “단어의 의미는 주변 단어로 결정된다”는 **분포 가설**(distributional hypothesis)을 학습으로 구현한 것.

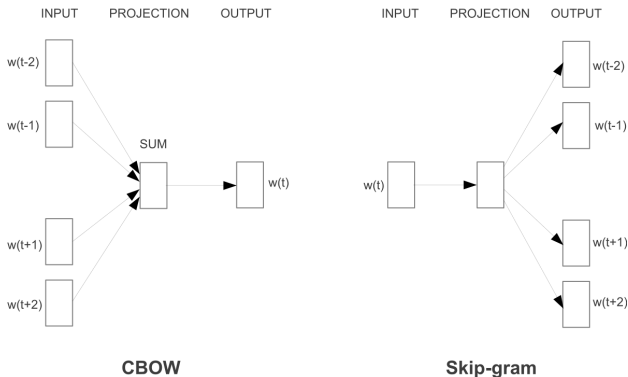
같은 목표

고차원(또는 벡터가 아닌) 원본을, **유용한 구조를 보존한 채** 저차원 벡터로 압축.

함께 자주 나타나는 것들은 가까운 벡터로 모인다. — 이 장 전체의 한 줄 원칙.

skip-gram: “다음 단어 맞추기”는 수단일 뿐

- **Skip-gram**: 중심 단어 하나로 주변 단어들을 예측하도록 작은 신경망을 학습.
- 학습이 끝나면 **신경망의 가중치 자체가 각 단어의 임베딩 벡터**가 된다.
- “다음 단어 맞추기”는 수단일 뿐 — 진짜 산출물은 그 부산물인 벡터.



학습된 벡터의 놀라운 성질

$$\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$$

- “왕에서 남자다움을 빼고 여자다움을 더하면 여왕” — 학습 당시 **전혀 명시적으로 가르치지 않았는데도** 성립.
- 벡터 공간의 **기하학적 구조**가 스스로 그렇게 정리된 것.

(Mikolov, Sutskever, Chen, Corrado & Dean, 2013 — 단어 유사도·유추(analogy) 분석을 본격 도입한 논문.)

50년의 여정: 단어를 숫자로 만들다

연도	인물	내용
1958	Miller & Chapman	“단어는 비슷한 맥락 에 함께 등장하는 것끼리 서로를 정의한다”.
1965	J. Firth	“You shall know a word by the company it keeps.” (그 단어의 친구를 보면 그 단어를 안다) — 분포 가설 . 기계로 학습시킬 수단은 아직 없음.
1988	Hofmann & Schütze	LSI (잠재 의미 인덱싱) — “단어×문서” 빈도 행렬에 PCA와 같은 분해 를 적용. 단, 선형 분해라 의미 유사도 연산까지는 못 만듦.
2003	Bengio et al.	신경망 언어모델 — “단어 벡터를 신경망 학습으로 만든다”가 실용화.
2013	Mikolov et al.	word2vec — 효율화를 끝마침.

“단어 → one-hot”이 아니라, “단어 → 학습된 저차원 밀집(dense) 벡터”로: 한 단어의 벡터 = 그 단어가 등장한 문맥들의 압축 요약.

bag-of-words의 두 가지 문제

그 전에는 **bag-of-words**가 표준 — 어휘의 모든 단어를 나열해 각 문장마다 **단어가 몇 번 등장하는지 세어서** 벡터로. (예: “고양이는 귀엽다” $\rightarrow [1, 1, 0, \dots, 0]$)

- ① **차원이 어휘 크기와 같음** — 10만 단어 어휘면 10만 차원, 99.99%가 0인 **극도로 희소**(sparse)한 벡터. 14.1의 PCA가 이런 데이터에 효과적인 것도, “1인 성분이 몇 개뿐”인 구조를 알아서 압축해 주기 때문.
- ② **단어 간 “거리”에 의미가 없음** — one-hot에서는 “고양이”와 “강아지”도 “고양이”와 “사과”와 **완전히 같은 거리**(내적 0, 코사인 0). 의미를 담은 공간이 애초에 설계되지 않은 것.

skip-gram의 실제 구조

“작은 신경망”을 정확히 보면:

층	크기
---	----

입력층	어휘 크기 V 개 (one-hot 입력)
-----	----------------------------------

은닉층	d 개 (= 임베딩 차원)
-----	--------------------------

출력층	다시 어휘 크기 V 개
-----	----------------

- 중심 단어 c 의 임베딩 = $\mathbf{v}_c$ (은닉층의 “가중치 행”). 출력층에서 단어 t 를 가리키는 벡터 = $\mathbf{u}_t$.
- “중심 c 의 바로 옆에 t 가 나올 확률” (2.3절 시그모이드):

$$p(t \mid c) = \sigma(\mathbf{u}_t^\top \mathbf{v}_c) = \frac{1}{1 + e^{-\mathbf{u}_t^\top \mathbf{v}_c}}$$

즉, 두 벡터의 내적이 클수록 t 가 c 옆에 나올 가능성이 크다고 학습됨.

학습 데이터와 loss

- 학습 데이터: 말뭉치 전체에서 **모든 (중심, 주변) 단어 쌍** — 앞뒤 w 개 **윈도우** 안에서 실제로 함께 나온 쌍.
- loss: 3절 다중분류로 쓴 바로 그 **이진 교차엔트로피**를 쌍마다 더해줌:

$$\mathcal{L} = - \sum_{(c,t) \in \text{말뭉치}} \left[\log p(t \mid c) + \sum_{t' \in \text{네거티브 샘플}} \log(1 - p(t' \mid c)) \right]$$

한 줄

“함께 나오는 쌍의 내적을 **크게**, 안 나오는 쌍의 내적을 **작게**” — 이 상대적 **순위(ranking)**를 만드는 것이 word2vec 전체의 학습 목표.

CBOW vs Skip-gram

CBOW (Continuous Bag-of-Words)

- **주변** → **중심** 방향.
- 주변 단어 여러 개의 임베딩을 **평균**해서 중심 단어를 예측.
- 계산이 **빠름**.
- **고빈도 단어**(주변에 자주 나오는 명사·동사)의 의미를 더 정확히 잡음.

Skip-gram

- **중심** → **주변** 방향.
- 하나의 중심에서 주변을 **하나씩** 예측.
- 계산은 **느림**.
- **저빈도 단어**(희귀 고유명사, 전문 용어)를 더 정확히 잡음 — “주변이 무엇이었는가”를 하나하나 명시적으로 학습하기 때문.

실용적 합의

데이터가 크고 **빨리** 뽑고 싶으면 CBOW, **희귀단어의 정확도**가 중요하면 Skip-gram. (본문의 코드는 Skip-gram)

네거티브 샘플링: softmax를 피하는 트릭

- 위 loss를 그대로 쓰면, 각 쌍마다 **어휘 전체 V 개** 확률을 전부 계산해야 한다(softmax의 분모).
- $V = 10^6$ 이면 매 스텝 100만개의 시그모이드 — 수십억 쌍이 모이면 **불가능**.
- **네거티브 샘플링**: 정답 t 하나를 맞히는 대신, 무작위 k 개의 “가짜 단어” $t'_1, \dots, t'_k$ (이 k 를 k_{neg} , 코드에선 neg_k)도 함께 “아니다”로 학습.
- 즉, 하나의 쌍에 대해 $1 + k_{\text{neg}}$ 개의 **이진 분류**를 동시에 — 1개만 label=1(정말 옆에 나온 단어), 나머지 label=0(가짜).

효과

“어휘 전체 비교” softmax ($O(V)$) $\rightarrow$ “정답 1개 + 가짜 k_{neg} 개 비교” 이진 분류 ($O(k_{\text{neg}})$).
 $k_{\text{neg}} = 5$ 정도면 충분 (원 논문: $V \approx 10^7$ 일 때 5, $V \approx 10^5$ 이하면 2~5).

skip-gram 학습 코어 (핵심만)

```
def train_skipgram(corpus, window=2, dim=8, epochs=50, lr=0.05, neg_k=3):
    vocab = sorted(set(corpus))
    idx = {w: i for i, w in enumerate(vocab)}
    V = len(vocab)
    W_in = [[random.uniform(-0.5, 0.5) for _ in range(dim)] for _ in range(V)]
    W_out = [[random.uniform(-0.5, 0.5) for _ in range(dim)] for _ in range(V)]

    pairs = []
    for i, center in enumerate(corpus):
        for j in range(max(0, i - window), min(len(corpus), i + window + 1)):
            if j != i:
                pairs.append((idx[center], idx[corpus[j]]))

    for _ in range(epochs):
        random.shuffle(pairs)
        for c, o in pairs:
            targets = [(o, 1)] + [(random.randrange(V), 0) for _ in range(neg_k)]
            for t, label in targets:
                z = sum(W_in[c][k] * W_out[t][k] for k in range(dim))
                pred = sigmoid(z)
                grad = (pred - label) * lr
                for k in range(dim):
                    W_in[c][k] -= grad * W_out[t][k]
                    W_out[t][k] -= grad * W_in[c][k]
```

그래디언트가 로지스틱 회귀와 똑같은 이유

- 이 학습의 그래디언트도 ($\text{pred} - \text{label}$)이라는 **똑같은 인자**로 시작 — 2장 로지스틱 회귀의 ($h_w(x) - y$)와 **완전히 같은 형태**.
- **우연이 아님**: word2vec을 “중심 c 옆에 t 가 실제로 나왔는가($\text{label}=1$) / 무작위 가짜인가($\text{label}=0$)”를 맞추는 **이진 분류** 문제로 학습시키기 때문.
- “가짜를 많이 넣으면 더 정확해지지 않나?” — **항상 좋은 것은 아님**:
- $k_{\text{neg}} \approx 0$: “함께 나오는 쌍의 내적만” 커지고 안 나오는 쌍엔 페널티가 없어 **모든 내적이 비슷해짐** (비슷하지 않은 단어까지 코사인이 높아짐).
- k_{neg} 매우 큼: 한 쌍당 이진 문제가 너무 많아 **학습 신호가 희석**되고 학습도 느려짐.

핵심: 네거티브 샘플링은 “softmax를 근사”하는 것이 아니라, **softmax 대신 다른 loss**(이진 분류)를 쓰는 것 — 매 스텝 $O(V)$ 인 계산 비용(대체, loss substitution)을 피하기 위한 것. 이 절의 작은 말뭉치($V = 10$)에서는 $k_{\text{neg}} = 4$ 가 좋음.

손으로 한 번: 한 스텝 갱신 따라가기

임베딩 차원 $d = 2$, 학습률 $\eta = 0.1$, **네거티브 샘플** $k = 0$ (가짜 없음), 쌍 $(c, t) = (\text{고양이}, \text{귀엽다})$, label=1. 초기:

$$\mathbf{v}_{\text{고양이}} = (0.4, -0.2), \quad \mathbf{u}_{\text{귀엽다}} = (0.1, 0.3)$$

Step 1 — 시그모이드. 내적 $z = 0.1 \times 0.4 + 0.3 \times (-0.2) = 0.04 - 0.06 = -0.02$.
 $\sigma(-0.02) \approx 0.495$ — 동전 던지기 수준의 예측.

Step 2 — 그래디언트. 2.3절 로지스틱 회귀와 **완전히 같은** 형태:

$$\text{err} = \sigma(z) - y = 0.495 - 1 = -0.505$$

벡터 갱신: 내적이 0.495 $\rightarrow$ 0.499로

Step 2 (계속) — 업데이트:

$$\Delta \mathbf{v}_c = -\eta \cdot \text{err} \cdot \mathbf{u}_t = -0.1 \times (-0.505) \times (0.1, 0.3) = (0.00505, 0.01515)$$

$$\Delta \mathbf{u}_t = -\eta \cdot \text{err} \cdot \mathbf{v}_c = (0.02020, -0.01010)$$

Step 3 — 새 값: $\mathbf{v}'_{\text{고양이}} = (0.4051, -0.1849)$, $\mathbf{u}'_{\text{귀엽다}} = (0.1202, 0.2899)$.

무엇이 달라졌는가

새 내적 $z' = 0.1202 \times 0.4051 + 0.2899 \times (-0.1849) = -0.0049$ — 이전 -0.02 보다 **0에**

가까워짐, $\sigma(z') \approx 0.4987 > 0.495$.

$k_{\text{neg}} = 4$ 의 실제 훈련에서는 이 “정답 1쌍” 업데이트에 “가짜 4쌍”(err = $\sigma(z) > 0$ 이라 **내적을 낮추는** 방향)이 동시에 더해진다.

실습: 코사인 유사도로 비슷한 단어 찾기

동물·과일 관련 작은 말뭉치로 학습 후, “고양이는”과 가장 비슷한 단어:

```
def cos_sim(a, b):  
    dot = sum(x*y for x, y in zip(a, b))  
    na = math.sqrt(sum(x*x for x in a))  
    nb = math.sqrt(sum(x*x for x in b))  
    return dot / (na*nb + 1e-9)  
  
corpus = ( "고양이는 동물이다강아지는동물이다사자는동물이다"  
           "고양이는 귀엽다강아지는귀엽다"  
           "사과는 과일이다바나나는과일이다포도는과일이다"  
           "사과는 맛있다바나나는맛있다" ).split()  
random.seed(42)  
emb = train_skipgram(corpus, window=2, dim=8, epochs=200, lr=0.1, neg_k=4)  
query = "고양이는"  
sims = sorted(((w, cos_sim(emb[query], emb[w])) for w in emb if w != query),  
               key=lambda kv: -kv[1])
```

결과: 동물이다 0.875, 사자는 0.622, 귀엽다 0.581, **사과는 0.549**, 강아지는 0.533, 과일이다 0.287, ...

결과 읽기: 문맥이 비슷한 단어가 가깝다

- “동물” 관련(동물이다, 사자는, 강아지는)이 “과일” 관련 (과일이다, 포도는, 바나나는)보다 **대체로** 코사인 유사도가 높음.
- “고양이는”이 실제로 등장한 문맥(동물 문장)과 비슷한 문맥에 나온 단어일수록 **벡터 공간에서도 가까움**.
- 말뭉치가 아주 작아 완벽하진 않다(예: “사과는”가 예상보다 높게) — 데이터가 수십억 단어로 커지면 노이즈가 줄고, “의미적 유사도 = 벡터의 각도”가 선명해진다.

왜 코사인 유사도인가 (14.1 PCA와 같은 이유)

벡터의 “**방향**”이 의미이고, “**길이**”는 (주로) **빈도**라서, 비슷한 단어를 찾을 때 각도(= 코사인)를 보고 길이는 무시하는 것이 맞다.

왜 길이가 제각각인가: 빈도 $\propto$ 노름

- 위 임베딩에서 단어 벡터의 노름: “사자는”=2.88, “맛있다”=2.40, “고양이는”=2.39, “사과는”=2.00, “귀엽다”=1.93 — 같은 “단어”라도 길이가 제각각.
- **기대된 결과:** 빈도가 높은 단어가 훈련에서 **더 많이 업데이트**되어 벡터가 더 길어진다.
- 실전 word2vec의 후처리에서 임베딩을 **단위벡터로 정규화**하는 것이 이 효과를 제거하는 표준 방법 (Lin & Mikolov, 2015).

노름은 “빈도”가 아니라 “의미”가 아니다 — 그래서 유사도 계산은 각도(코사인)로.

“왕 – 남자 + 여자 = 여왕”의 기하학

- 학습 목표(“함께 나오는 쌍의 내적을 크게, 안 나오는 쌍을 작게”)가 **의미 축이 서로 직교**하게 정렬되도록 압력을 넣는다.
- 예: “왕/여왕”을 구분하는 **성별 축**과 “왕/신하”를 구분하는 **지위 축**이 서로 다른 방향 → “왕에서 남자다움 빼고(= 성별 축을 따라 여왕 쪽으로 이동) 여자다움 더하면” 여왕에 도달.
- 이 **병렬 4각형**(parallelogram) 구조가, 두 축이 직교하고 **각 축의 스케일이 일관될 때** 기하학적으로 성립.

“스케일이 일관된다”는 뜻

“남자 → 왕” 벡터와 “여자 → 여왕” 벡터의 **크기가 비슷**해야 병렬 4각형이 닫힌다.

학습 데이터에서 왕/여왕이 비슷한 문맥(“왕이 등극하다” / “여왕이 등극하다”)에서 비슷한 **빈도**로 나오면 스케일이 비슷 → 4각형이 닫힘. 반대로 빈도 차이가 극단적(왕 10만 번, 여왕 100 번)이면 **노름 차이**가 커져 4각형이 닫히지 않는다.

(실제 king/queen 예가 잘 되는 것 = Google News 60억 단어 말뭉치에서 빈도비가 극단적이지 않아서.)

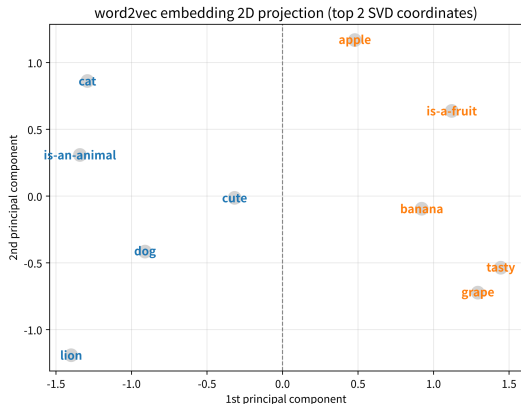
실습: 축 전환 — “고양이는 — 귀엽다 + 맛있다”

```
target = [a - b + c for a, b, c in zip(emb["고양이는"], emb["귀엽다"], emb["맛있다"])]
analogy = sorted(((w, cos_sim(target, emb[w])) for w in emb
                  if w not in {"고양이는", "귀엽다", "맛있다"}),
                 key=lambda kv: -kv[1])[:3]
print(analogy)
# 사과는(' ', 0.571), 바나나는(' ', 0.478), 과일이다(' ', 0.456)]
```

- “귀엽다”는 **동물-속성 축**, “맛있다”는 **과일-속성 축**의 한 방향 — 뽀렘/덧셈이 “**동물-속성** → **과일-속성**”으로의 축 전환과 같음.
- 10개 단어짜리 말뭉치에서 0.571로 “사과”가 1위 — “의미 축의 병렬 이동”이 이미 작은 스케일에서 학습되고 있음을 보여주는 좋은 신호.

병렬 4각형이 성립하려면 d 가 “포착할 의미 축의 수”보다 **충분히 커야** 한다 — 성별·지위·시제·수·품사가 10개인데 $d = 8$ 이면 몇몇 축이 “섞여” 정확히 닫히지 않음. = 14.1 PCA와 같은 **차원-정확도 트레이드오프**.

2차원 투영: 동물 vs 과일 두 군집



10개 단어 임베딩을 8차원 → 2차원(SVD 상위 2 좌표)으로 투영.

- 10개 단어가 “동물”/“과일” 두 군집으로 **거의** 나뉨 — 왼쪽:
고양이는·강아지는·사자는·동물이다·귀엽다,
오른쪽:
사과는·바나나는·포도는·과일이다·맛있다.
- “사과는”가 과일 군집에 완전히 속지 않고 위쪽으로 튀어 나온 것 = “사과는”가 “맛있다”와 **두 번** 함께 나온 반면 바나나는/포도는 한 번 → **빈도 차이** → 노름 차이 → 위치 차이.

말뭉치가 작을수록 이 **빈도 효과**가 군집 구조를 어지럽힌다 — “문맥이 비슷한 단어가 벡터 공간에서 가깝다”는 핵심 성질이 이미 작은 스케일에서 확인되는 것.

자주 하는 실수: “벡터 길이가 모두 비슷하다”고

- “임베딩 벡터는 모두 **같은 길이**(노름)라 코사인으로 비교해도 된다”는 믿음은 틀림 — word2vec 임베딩의 **노름은 서로 다름**(위 실험: 사자는=2.88, 사과는=2.00).
- 원인: 학습 중 **빈도가 높은 단어가 더 많이 업데이트**되어 길어짐 (빈도 $\propto$ 노름의 경향).
- **코사인 유사도**는 노름을 제거한 **각도만** 측정 — “의미 유사도”를 재기엔 적합.
- 그러나 **유사도 연산**($\mathbf{a} - \mathbf{b} + \mathbf{c}$) 할 때는, **노름 차이가 병렬 4각형의 “닫힘”에 영향**을 준다는 점을 기억 — “왕 - 남자 + 여자”가 정확히 여왕에 닿으려면 “남자→왕”과 “여자→여왕”의 **노름이 비슷**해야 한다.

실용적 정리

비슷한 단어 찾기: 코사인 유사도 (노름 무시) / **유사도 연산(analogy)**: 노름 차이를 주의.
실전에서는 **단위벡터로 정규화**한 뒤 analogy — “빈도 효과” 제거로 정밀도 향상.

자주 하는 실수: “window가 클수록 좋아”

window = 중심 단어의 **앞뒤 몇 개**를 “주변”으로 취할지 — 임베딩이 “**국소적** 의미”를 잡을지 “**글로벌** 의미”를 잡을지 결정.

작은 window (1~2)

- “바로 옆 단어”만 봄.
- **구체적·국소적** 의미를 선명히 — 예: “먹다”의 목적어 (“사과를 먹다” vs “밥을 먹다”).

큰 window (10~15)

- 멀리 떨어진 단어까지 포함.
- **주제적·글로벌** 의미를 잘 — 예: “경제”, “정책” 같은 상위 주제어.

정리

“큰 window가 더 좋다”는 믿음은 **틀림** — window가 크면 “바로 옆”의 구체적 신호가 먼 단어의 노이즈에 묻혀 **구체적 의미 해상도가 떨어짐**. 반대로 너무 작으면 주제를 못 포착.
원 논문 실험: **window=5**가 많은 어휘에서 균형이 좋음; **저빈도 단어**엔 큰 window, **고빈도 단어**엔 작은 window가 좋음.

window=1 vs 2를 말뭉치로 재보기

“사자는”(동물)과 “바나나는”(과일)의 코사인 유사도 **차이** (= “동물-과일 구분력”)를 window별로 측정:

window = 1 : gap = +0.303 window = 2 : gap = +0.341

- window=2가 **약간** 더 좋은 구분력 — 이 작은 말뭉치에서는 window=2가 “고양이는 동물이다”처럼 **세 단어 패턴**을 포착할 수 있어서(1이면 “고양이는-동물이다”만, 2면 “고양이는-동물이다-강아지는”도) 문맥 정보가 조금 더 담김.
- window를 5, 10으로 키워도 **더 좋은 결과는 안 나옴** — 이 작은 말뭉치(고유어 10개, 16 토큰)에선.
- 이 실험이 보여주는 것: “window 최적값”은 **말뭉치의 길이와 어휘 크기**에 따라 달라진다.

확인 문제 (1/2)

- **1. (시그모이드 손계산)** 초기 $\mathbf{v}_c = (0.4, -0.2)$, $\mathbf{u}_t = (0.1, 0.3)$ 일 때: (a) $\sigma(z)$, (b) label=1일 때 $\text{err} = \sigma(z) - 1$, (c) $\eta = 0.1$ 일 때 $\mathbf{v}_c$ 의 첫 성분 업데이트량 $\Delta v_{c,1}$.
답: $z = -0.02$, $\sigma \approx 0.495$, $\text{err} \approx -0.505$, $\Delta v_{c,1} = -\eta \cdot \text{err} \cdot u_{t,1} = -0.1 \times (-0.505) \times 0.1 \approx +0.00505$.
- **2. (네거티브 샘플의 역할)** $\text{neg_k} = 0$ (정답만 학습)으로 바꾸면 코사인 유사도 분포가 어떻게 변하나?

힌트: 함께 나오는 쌍의 내적을 **올리는** 학습만 있고, 안 나오는 쌍의 내적을 **낮추는** 학습은 **전혀 없음** →
“고양이는-사자는”이 +0.928으로 극단 상승, “고양이는-바나나는”는 +0.282 → -0.081으로 오히려 **음수**
(-0.081)로 떨어짐 — 공유 문맥(“동물이다”)을 통해 연결된 동물 단어들이 서로 거의 같은 방향으로 수렴. “동물-과일 gap”이 1.009로 “크게 보이지만” 이는 “과일이 밀려난” 것이 아니라 “동물 쪽이 극단으로 올라간” 결과 — **상대적 순위(ranking) 전체를 읽어야 하는 함정.**

확인 문제 (2/2) + 자주 묻는 질문

- **3. (CBOW vs Skip-gram)** 100만 어휘 · 10억 토큰 뉴스 말뭉치에서 **희귀한 인명·지명**(수십 번씩만 등장)의 임베딩 품질이 최우선이라면?

Skip-gram — 희귀 단어 1개가 중심 역할을 할 때, 그 주변을 **하나씩** 예측하는 훈련이 $2w$ 개 (앞뒤 window)만큼 수행되므로, 고빈도 중심의 CBOW보다 희귀 단어에 **상대적으로 더 많은** 학습 신호가 도달.

- **Q. dim은 어떻게?** 14.1 PCA와 같은 트레이드오프 — 작으면 미묘한 차이(편향)를 못 담고, 크면 데이터가 늘고 과적합(분산). 실전은 **100~300차원**.
- **Q. LLM 토큰 임베딩과 같은 건가?** 뿌리는 같으나 word2vec은 임베딩 **자신이 목표**이고 window 고정; LLM은 임베딩이 Transformer의 **첫 층일 뿐** — 어텐션을 거치며 **문맥에 따라 계속 재해석**됨 (같은 단어도 문장에 따라 다른 벡터).
- **Q. “의미를 담고 있다” vs PCA의 “분산을 담고 있다”?** 압축의 **기준**이 다름 — PCA: **데이터 내적**(분산 최대, 라벨 무시, 선형), word2vec: **예측 성능**. 둘 다 비지도이나 “무엇을 보존하는가”의 정의가 다름.

자주 묻는 질문 (계속)

- **Q. 단위벡터로 정규화하면 무조건 좋아지나? 비슷한 단어 찾기**(코사인 유사도)에는 **일반적으로 좋음**
— “빈도 효과”(노름 차이) 제거로 의미 유사도만 남김.
그러나 **유사도 연산**($\mathbf{a} - \mathbf{b} + \mathbf{c}$)에는 **반드시 좋은 것은 아님** — 정규화가 “의미 축의 스케일 정보”를 지우기 때문: “왕 → 여왕” 이동 벡터와 “남자 → 여자” 이동 벡터의 **크기 비율**이 의미축의 상대적 중요도를 담을 수 있는데, 정규화하면 이 정보도 사라짐.
- **실전:** 정규화 **전/후 두 버전**을 모두 만들어 보고, **용도**(similar-word search vs analogy)에 맞춰 선택하는 것이 정답.

14.3 임베딩을 실전에 쓰기: Transformer 임베딩과 RAG

Transformer의 첫 층도 word2vec의 아이디어

12장에서 어텐션을 배울 때 각 단어는 이미 “임베딩 벡터”로 주어졌다고 **가정**하고 시작했다 — 그 임베딩의 정체는:

학습 가능한 임베딩 테이블

Transformer의 **맨 첫 층** = 토큰 ID → 고정 차원 벡터를 매핑하는 **lookup 테이블**.

word2vec: “중심 단어로 주변을 맞추기”를 **독립적 목표**로 학습. Transformer 토큰 임베딩: “다음 토큰 맞추기” 과제 전체를 풀면서 **부산물**로 함께 학습.

목표는 다르지만, “그 벡터가 있으면 과제를 더 잘 풀 수 있으니까 학습된다”는 **메커니즘은 같다**.

두 가지 차이: 고정이나 재해석이나

차이 1: 문맥 고정 vs 재해석

- word2vec: 벡터는 **문맥과 무관하게 고정**
— 같은 단어는 **항상** 같은 벡터.
- Transformer: 토큰 임베딩은 뒤의 **어텐션**
층들을 거치며 문맥에 따라 계속 재해석 —
“은행”의 초기 임베딩은 하나지만, 문장에
따라 최종 표현은 **다름**.

차이 2: 산출물이나 조각이나

- word2vec: 벡터 자체가 **최종 산출물**.
- Transformer: 토큰 임베딩은 **훨씬 큰**
모델의 한 조각일 뿐.

12장에서 “가정”했던 것이, 14.2의 word2vec과 같은 정신으로 “학습된 임베딩”이었다.

RAG: 임베딩 공간에서 “가까운 문서”를 찾는다

- **RAG**(Retrieval-Augmented Generation)를 프롬프팅 기법 으로 보면(13장) “관련 문서를 프롬프트에 붙여준다” — **어떻게 찾나**는 순전히 이 장의 문제:
- **질문도 벡터, 문서(조각)도 벡터** → **코사인 유사도(또는 내적)**가 가장 큰 문서를 찾는다.
- 14.2에서 **단어** 벡터의 유사도를 코사인으로 잰던 것과 **정확히 같은 연산** — 대상이 단어에서 문서로 바뀐 것뿐.

밀집 검색(dense retrieval) vs 키워드 매칭

“임베딩 → 유사도 검색” 파이프라인 = **밀집 검색**.

키워드 매칭(TF-IDF, BM25): “정확히 같은 단어가 있는가”. **밀집 검색**: “의미가 비슷한가”를 벡터 공간 에서 잰다 — 질문에 없는 단어로 쓰인 문서도 찾음 (“자동차 연비”로 검색 → “1km당 연료 소비량” 문서가 걸림).

RAG의 검색 단계 원 논문: Lewis et al., 2020 — 검색 단계를 희소 인덱스(BM25 등)로 하고 **생성 단계**까지 연결한 것.

RAG 검색 파이프라인을 한 줄로

질문 $\xrightarrow{\text{임베딩}}$ 질문 벡터 q

문서 $d \xrightarrow{\text{임베딩}}$ 문서 벡터 d

$\cos(q, d)$ 계산 $\rightarrow$ 유사도 큰 상위 1~k 개 문서 선택

핵심 원리

질문도 벡터, 문서도 벡터, 가까운 것을 검색.

“의미가 비슷한가”를 벡터 공간의 **각도**로 잴다는 것이 전부 — 대상이 단어(14.2)에서 문서로 바뀐 것뿐.

이 한 줄이 2차원 손계산 $\rightarrow$ 20줄 데모 $\rightarrow$ 실전 수십억 문서까지 **같은 구조**로 확장된다.

손으로 한 번: 2D 밀집 검색

차원 768을 2로 낮춘 예. 질문 “자동차 연비” $\mathbf{q} = (4, 1)$, 문서:

- $\mathbf{d}_1 = (3, 2)$: “차량의 연료 소비량은 1km당 소모 연료로 측정한다”
- $\mathbf{d}_2 = (2, 3)$: “자동차 보험을 고르는 법”
- $\mathbf{d}_3 = (1, 3)$: “양자 컴퓨터는 정보를 어떻게 처리하는가”

$\mathbf{q}$ 와 $\mathbf{d}_1$ 은 단어를 하나도 공유하지 않는다(자동차/차량, 연비/연료 소비량) — 키워드 매칭이 못 찾는 경우.

1단계 — 내적: $\mathbf{q} \cdot \mathbf{d}_1 = 4 \times 3 + 1 \times 2 = \mathbf{14}$, $\mathbf{q} \cdot \mathbf{d}_2 = 11$, $\mathbf{q} \cdot \mathbf{d}_3 = 7$.

2단계 — 노름: $\|\mathbf{q}\| = \sqrt{17} \approx 4.12$, $\|\mathbf{d}_1\| = \|\mathbf{d}_2\| = \sqrt{13} \approx 3.61$, $\|\mathbf{d}_3\| = \sqrt{10} \approx 3.16$.

코사인 유사도로 순위 매기기

3단계 — 내적을 노름의 곱으로 나눔:

$$\cos(\mathbf{q}, \mathbf{d}_1) = \frac{14}{\sqrt{17}\sqrt{13}} = \frac{14}{\sqrt{221}} \approx \mathbf{0.942}$$

$$\cos(\mathbf{q}, \mathbf{d}_2) = \frac{11}{\sqrt{221}} \approx \mathbf{0.740}, \quad \cos(\mathbf{q}, \mathbf{d}_3) = \frac{7}{\sqrt{170}} \approx \mathbf{0.537}$$

순위: $\mathbf{d}_1$ (0.942) > $\mathbf{d}_2$ (0.740) > $\mathbf{d}_3$ (0.537)

1위는 단어를 하나도 공유하지 않는 문서 1 “차량의 연료 소비량...”. 반면 “자동차”라는 **단어**가 포함된 문서 2는 2위에 그침.

키워드 매칭이라면 문서 2만 걸리고 문서 1은 **아예 놓쳤을** 것.

차원이 2라는 것만 빼면 768차원도 계산은 **동일** — “각도가 작을수록 의미 비슷”은 2차원과 768차원에서 같다. 이 연산은 14.2의 단어 유사도와 **정확히 같음** (단어 → 문서).

실습: 미니 RAG — 문서 벡터 만들기

14.2의 `train_skipgram.cos_sim`을 그대로 쓰고, 문서 안 단어 벡터를 **평균**해 “문서 벡터”를 만드는 `mean_vec`를 추가. 에너지·동물·컴퓨팅 5개 문서:

```
# 문서벡터 = 존재하는단어벡터의평균
def mean_vec(words, emb):
    dim = len(next(iter(emb.values())))
    present = [w for w in words if w in emb]
    if not present:
        return None
    v = [0.0] * dim
    for w in present:
        for k in range(dim):
            v[k] += emb[w][k]
    return [x / len(present) for x in v]

tok = lambda s: s.replace(".", " ").split()
corpus_docs = {
    "D1 전기차": "전기차는 배터리로움직인다 . . .",
    "D2 태양광": "태양광 발전은햇빛으로전기를만든다 . . .",
    "D3 양자": "양자 컴퓨터는큐비트로정보를처리한다 . . .",
    "D4 고양이": "고양이는 애묘인이좋아하는동물이다 . . .",
    "D5 배터리": "배터리와 태양광발전은 . . .",
}
flat = []
```

미니 RAG: 두 질문으로 검색

```
for q in ["양자 컴퓨터", "고양이는 손질한다"]:  
    qv = mean_vec(tok(q), emb)  
    sims = sorted(((n, cos_sim(qv, dv)) for n, dv in docvecs.items()),  
                  key=lambda kv: -kv[1])  
    print("\질문n:", q)  
    for n, s in sims:  
        print(f"    {n}: {s:+.3f}")
```

질문: 양자 컴퓨터

질문: 고양이는 손질한다

D3 양자: +0.793 (1위)

D2 태양광: +0.454

D4 고양이: +0.408

D1 전기차: +0.354

D5 배터리: +0.244

D4 고양이: +0.951 (1위)

D5 배터리: +0.558

D3 양자: +0.555

D2 태양광: +0.481

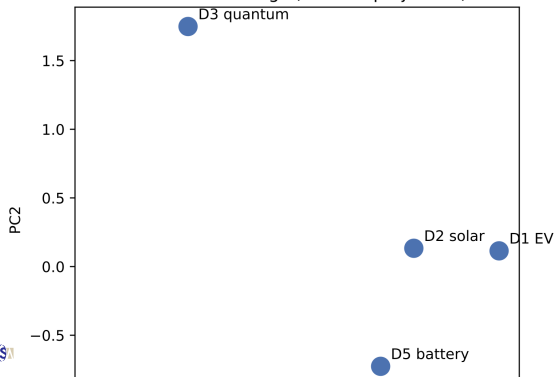
D1 전기차: +0.329

random.seed(42)로 재현 가능.

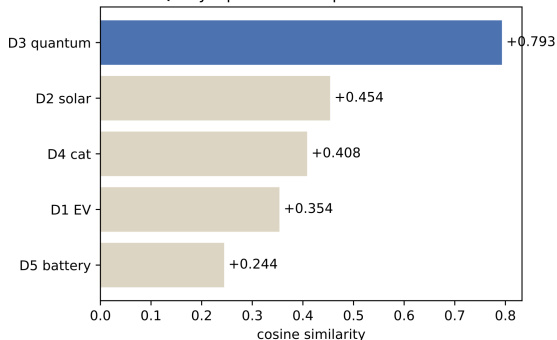
검색 결과 읽기: 선명한 1위

- “양자 컴퓨터”: 승자 D3 양자(0.793), 나머지 전부 0.46 아래 — 1위와 2위 사이에 **0.34**라는 깨끗한 간격이 열림.
- “고양이는 손질한다”: D4(0.951)가 압도적.
- “1위가 선명하게 분리된다” = 이 문서가 정답이라는 신호를 검색 단계에서 줬음 — RAG는 이 상위 1~k개를 프롬프트에 붙여줄 것.

5 doc embeddings (2D SVD projection)



Query "quantum computer" vs 5 docs



데모의 두 가지 한계 (의도된 것)

- **첫째**, mean_vec가 **문서의 단어 순서를 버린다** — “양자 컴퓨터”와 “컴퓨터 양자”는 **같은 문서 벡터**. 의도된 단순화: 의미 유사도 검색은 “어떤 단어가 들어 있는가”를 보고 순서는 잘 보지 않음 (14.2 bag-of-words 한계 와 같은 이야기).
단어 순서까지 살리려면: 12장 어텐션을 문서 단위로 돌리는 **문서 임베딩 모델**(실전 RAG가 하는 일).
- **둘째**, 이 데모는 RAG의 **검색 단계**만 — 검색된 문서를 받아 “이 문서에 기반해 답하라”고 하는 **생성 단계**는 LLM의 일이니 Ch. 17에서.

실전과의 차이: (a) train_skipgram 대신 **전문 임베딩 모델**, (b) 문서를 **문장 단위로 잘라** (chunking) 벡터화, (c) 수십억 문서에서 빠르게 최상을 찾게 **벡터 DB**(FAISS, HNSW 등). **원리는 지금 만든 이 20줄과 같다.**

자주 하는 실수: “내적 = 코사인 유사도”

“내적만 하면 되지 않나” 하고 $\mathbf{q} \cdot \mathbf{d}$ 로만 순위를 매기는 실수 — 둘은 **벡터의 길이가 다르면** 순서가 **달라진다** (14.2 “단어 벡터의 노름은 빈도와 비례”와 같은 구조).

위 손계산 예에서 문서 2의 벡터를 (4, 6)으로 바꿔보자 (= 같은 **방향**을 2배로 키운 것, 문서가 두

	$d_2 = (2, 3)$	$d'_2 = (4, 6) (= 2\text{배})$
배로 길어진 것과 같음):	내적 $\mathbf{q} \cdot \mathbf{d}_2$	$4 \times 4 + 1 \times 6 = 22$
	코사인	$\frac{22}{\sqrt{17}\sqrt{52}} \approx \mathbf{0.740}$

핵심

벡터를 2배로 키워도 코사인은 변하지 않음 — 분모의 노름도 2배가 되어 상쇄.

코사인은 **각도(방향)만**, 내적은 **각도 $\times$ 크기**를 봄 — “문서가 길면 내적이 커진다”는 **길이 편향**.

→ 의미 유사도 검색에서는 **코사인**(또는 단위벡터로 정규화한 뒤의 내적)을 쓴다 — 14.2 원칙의 **문서 버전**.

확인 문제 (1/2)

- **1. (코사인 손계산)** 2D 예에 $\mathbf{d}_4 = (1, 2)$ (“자동차 연비는 기름값에 비례한다”)를 추가. $\cos(\mathbf{q}, \mathbf{d}_4)$ 를 구하고 네 문서의 최종 순위를.

답: 내적 $\mathbf{q} \cdot \mathbf{d}_4 = 4 \times 1 + 1 \times 2 = 6$, $\|\mathbf{d}_4\| = \sqrt{5}$. $\cos = \frac{6}{\sqrt{17.5}} = \frac{6}{\sqrt{85}} \approx 0.651$. 순위:
 $\mathbf{d}_1 > \mathbf{d}_2 > \mathbf{d}_4 > \mathbf{d}_3$.

- **2. (내적 vs 코사인)** $(4, 6) = 2(2, 3)$ 이므로: (a) 내적이 $11 \rightarrow 22$ 로 두 배가 되는 이유, (b) 코사인이 0.740으로 그대로인 이유를 한 문장씩.

(a) 벡터가 정확히 2배이므로 $\mathbf{q}$ 와의 내적이 정확히 2배. (b) 노름도 2배 $\rightarrow$ 코사인의 분모(노름 곱)가 2배가 되어 상쇄.

확인 문제 (2/2): 확신도와 간격

- **3. (코드 검증)** 질문을 “고양이는 애묘인이 좋아하는” 으로: 1위 $D4=0.925$, 2위 $D3=0.654$ — 간격 0.271로, “양자 컴퓨터” 때(0.339)보다 **좁음**.

1~2위 간격 = 검색의 확신도

1위가 2위보다 **깨끗하게 멀리 떨어질**수록 “이 문서가 정답”이라는 신호가 강해진다.

간격이 좁을 때 실무 대응: **top- k** 를 늘려 후보를 여러 개 뽑거나, **임계값 아래 문서를 버리거나**, 아니면 “정답 문서가 없을 수 있다”고 **답을 거부**하는 판단을 한다.

자주 묻는 질문 — RAG (1/2)

- **Q. 문서 벡터를 왜 단어 벡터의 평균으로?** 14.2 bag-of-words와 같은 이유 — 문서를 “들어 있는 단어들의 **집합**”으로 보기 위함:
 - ▶ “양자 컴퓨터”와 “컴퓨터 양자”가 **같은 벡터**(단어 순서 무시).
 - ▶ 문서 **길이에 관계없이** 하나의 벡터가 나옴.

단점: **단어 순서·문법** 정보는 버려짐.

순서까지 살리려면: 12장 어텐션을 문서 전체에 돌리는 **문서 임베딩 모델**(실전 RAG가 쓰는 전문 임베딩 모델).

자주 묻는 질문 — RAG (2/2)

- **Q. 이 데모가 RAG 전체인가? 아니다 — 검색 단계만.** 검색된 문서를 받아 “이 문서에 기반해 답하라”고 하는 **생성 단계**는 LLM의 일이므로 Ch. 17에서.
- **Q. 왜 차원 8, 문서 5개처럼 작게?** 노트북에서 실행하고 **구조를 명확히** 보이려는 의도.

실전 RAG와의 차이 (구조는 동일)

- ① 임베딩: train_skipgram → **전문 임베딩 모델** (384~1536차원, “질문 ↔ 관련 문서” 쌍으로 학습).
- ② 문서: 통째 → **문장 단위로 잘라**(chunking) 벡터화.
- ③ 인덱스: **벡터 DB**(FAISS, HNSW 등)로 수십억 문서에서 빠르게 최상 탐색.

원리는 지금 만든 이 20줄과 같다 — 차원과 모델 규모만 다를 뿐.

이번 주차 정리

- ① **14.1 PCA** — “분산을 최대화하라”는 말과 “가장 큰 고유벡터를 찾아라”는 말은 **같은 문제** (라그랑주 승수법 $\rightarrow \Sigma v = \lambda v$). 주성분의 분산 = **고유값**, 버린 정보량 = **버린 고유값의 합**. 단위 다르다면 **반드시 표준화** 후 PCA.
- ② **14.2 word2vec** — 분포 가설(“친구를 보면 그 단어를 안다”)을 **이진 분류**(네거티브 샘플링)로 학습 — softmax $O(V)$ 를 $O(k_{\text{neg}})$ 로 대체. 학습된 벡터에서 **왕 - 남자 + 여자 $\approx$ 여왕**: 의미 축의 직교 + 스케일 일관으로 닫히는 병렬 4각형.
- ③ **14.3 임베딩 실전** — Transformer의 첫 층도 **학습된 임베딩 테이블**, RAG = **질문도 벡터, 문서도 벡터, 코사인 유사도로 가까운 것을 검색** — 14.2의 단어 유사도와 같은 연산.

다음 주 — Chapter 15. 잠재변수 생성모델

이 장은 관측 데이터 속에 숨은 저차원 구조를 **찾는** 쪽 — 15장은 그 구조(잠재 공간)가 데이터를 어떻게 **만들어내는가** 를 학습하는 생성 모델: **VAE $\rightarrow$ GAN · Diffusion**.